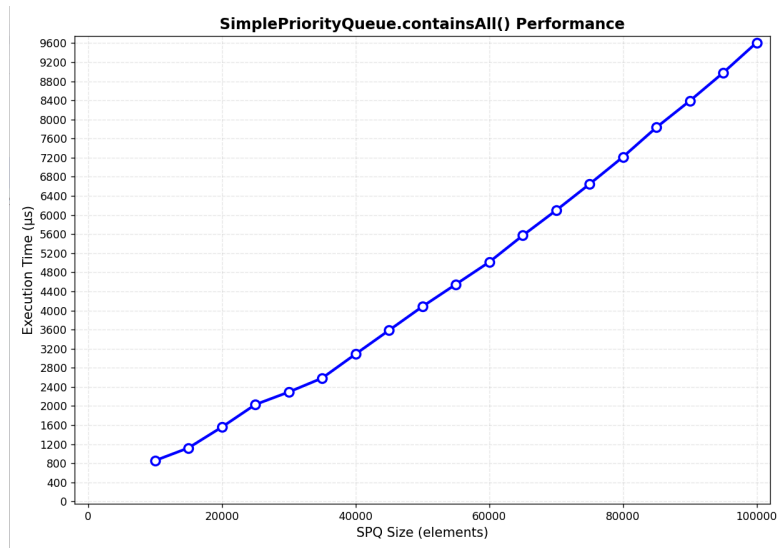


Assignment 3: SimplePriorityQueue

1. My programming partner is Zewei Ren and I submitted the program to Gradescope.
2.
 - 2.1. My programming partner and I switch roles about once an hour. I prefer to switch less often because I believe programming requires maintaining a consistent thought process, and switching too frequently makes it difficult for us to focus on the project.
 - 2.2. We applied pair programming techniques effectively in this assignment; I used these techniques to write about 60% of the code.
 - 2.3. I plan to spend more time thinking about edge testing during my next pair programming session.
 - 2.4. It took me about 3 hours to complete the programming and testing parts of this assignment.
 - 2.5. I felt that my programming partner and I communicated very frequently during the programming process, and I think he had a very positive attitude towards the assignment. I will continue to collaborate with him.
3. useJava ArrayListThis significantly reduces code volume during implementation; for example, it eliminates the need to manually write code for capacity checks and clearing, and elements can be automatically moved during insertion and deletion operations. In terms of development efficiency, using Java...ArrayListCan significantlyThis increases efficiency because we can directly call its internal methods to perform insertion, clearing, and deletion operations. In terms of runtime, there is little difference between the two, due to the underlying arrays and Java...ArrayListThe implementation methods are almost the same.
- 4.



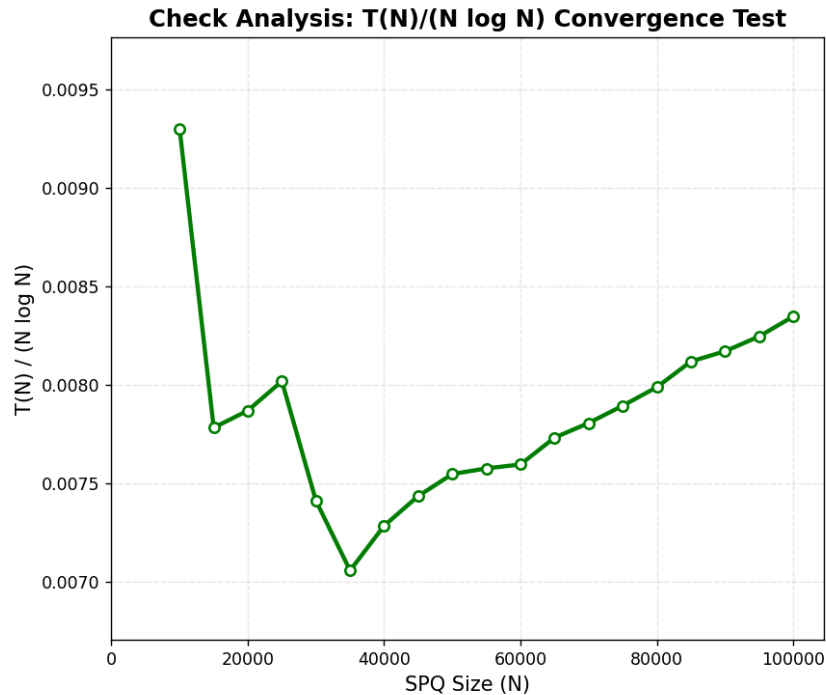
4.1.

4.2.

The settings for this graph are: x-axis represents the input size, y-axis represents the running time.

The chart shows how the runtime of the `containsAll()` method increases as the input size increases.

```
private static String problemSizeDescription = "SPQ size";
private static int problemSizeMin = 10000;      // Start with 10,000
elements
private static int problemSizeCount = 19;       // 19 data points total
private static int problemSizeStep = 5000;     // Increase by 5,000
each step
private static int experimentIterationCount = 20; // Run each size 20
times, take median
```



4.3.

The experiment measures the execution time of `containsAll()` as SPQ size increases from 10,000 to 100,000 elements in increments of 5,000. The `setupExperiment()` method populates both the priority queue and targets list with sorted integers, then shuffles the targets list to ensure random search order. The `runComputation()` method times only the `containsAll(targets)` call.

Results and Analysis:

The collected data shows execution times increasing from 814.7 μ s at $N=10,000$ to 10,170.1 μ s at $N=100,000$. Applying check analysis with $F(N) = N \log N$:

$N: 10,000$, $T(N)$ (μ s): 814.7, $N \log N$: 92,103.4, $T(N)/(N \log N)$: $8.84e-03$

$N: 50,000$, $T(N)$ (μ s): 4,313.0, $N \log N$: 587,496.6, $T(N)/(N \log N)$: $7.34e-03$

$N: 100,000$, $T(N)$ (μ s): 10,170.1, $N \log N$: 1,302,585.0, $T(N)/(N \log N)$: $7.81e-03$

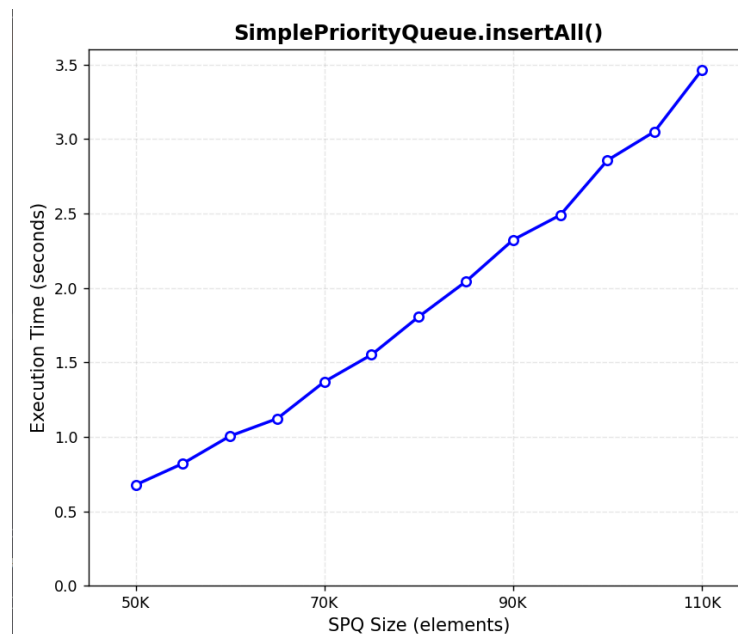
The ratio $T(N)/(N \log N)$ converges to approximately $7.8e-03 \pm 0.8e-03$, remaining relatively constant as N increases. According to check analysis principles:

If $T(N)/F(N)$ converges to a positive constant $\rightarrow F(N)$ correctly represents growth rate

Here, $T(N)/(N \log N) \approx 7.8e-03$ confirms $O(N \log N)$ complexity

4.4. I can deduce from this experiment that containsAllThe runtime behavior of the method is $O(N \log N)$.

5.

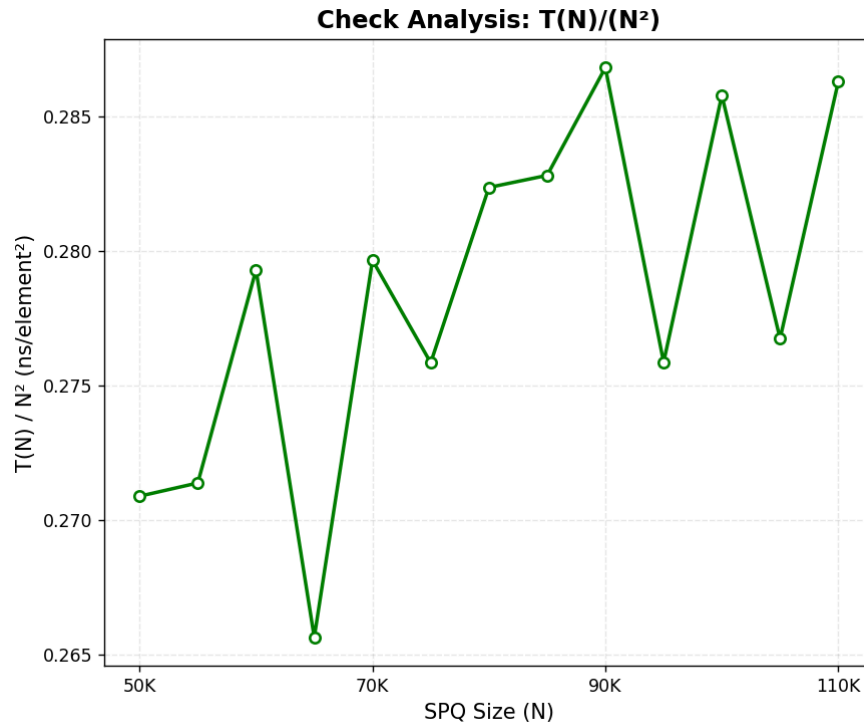


5.1.

5.2. The settings for this graph are: x-axis represents the input size, y-axis represents the running time.

The chart shows how the runtime of the `insertAll()` method increases as the input size increases.

```
private static String problemSizeDescription = "SPQ size";
private static int problemSizeMin = 50000;      // Start with 50,000
elements
private static int problemSizeCount = 15;       // 15 data points total
private static int problemSizeStep = 5000;     // Increase by 5,000
each step
private static int experimentIterationCount = 10; // Run each size 10
times, take median
```



5.3.

Applying check analysis with $F(N) = N^2$:

N: 50,000, T(N) (ns): 677,233,600, N^2 : $2.5e9$, $T(N)/N^2$: 0.271

N: 100,000, T(N) (ns): 2,857,882,900, N^2 : $1.0e10$, $T(N)/N^2$: 0.286

N: 110,000, T(N) (ns): 3,464,114,600, N^2 : $1.21e10$, $T(N)/N^2$: 0.286

The ratio $T(N)/N^2$ converges to approximately 0.286 ± 0.015 . According to check analysis:

$T(N)/N^2$ converges to positive constant 0.286

Therefore $O(N^2)$ correctly represents the growth rate

Inference about insert() method:

Since insertAll() calls insert() N times on an initially empty queue, and total time is $O(N^2)$, we can analyze the summation:

First insert: $O(1)$

Second insert: $O(\log 1 + 1) \approx O(1)$

i-th insert: $O(\log i + i/2) \approx O(i)$

Thus, individual insert() operations have:

Best case: $O(\log N)$ when inserting at the end

Average/Worst case: $O(N)$ due to element shifting

This explains the $O(N^2)$ behavior of insertAll()

Plot Description:

The insertAll plots show quadratic growth characteristics. The raw data plot displays a steep parabolic curve, while the check analysis plot

shows $T(N)/N^2$ converging to a constant value, visually confirming $O(N^2)$ complexity.

- 5.4. I can deduce from this experiment that `insertAllThe` method's runtime behavior is $O(N^2)$.